Game Design Document

Microjuego Fundamentos de Videojuegos

Miguel Ángel Jiménez San Nicolás
240096

1. Escenas:

Por simplificar, todo el juego se desarrolla en una única escena, llamada EscenaPrincipal. Esta escena contiene la cámara(fija), el espacio(el fondo), la nave principal, el canvas para las pantallas de pausa y muerte y los scripts de manejo de pooling.

2. Assets y GameObjects:

Para el desarrollo del juego he implementado 4 prefabs:

- Balas
- Meteoritos
- Fragmento de meteorito 1
- Fragmento de meteorito 2

En cuanto a GameObjects, se encuentran los siguientes:

1. Una cámara estática, con un aspect ratio Full HD(1920 x 1080) para que abarque la pantalla completa de un ordenador.

2. El fondo (Space_0): Es el espacio que se ve detrás de la nave, en él está anclado el script de generación de asteroides

3. La nave principal (Ship_0): Es la nave que controla el jugador, dentro lleva un GameObject con una posición ligeramente más adelantada de la punta. Este GameObject es el generador de balas desde donde dispara la nave. La nave lleva el script de Player para poder manejarla.

4. Un EventSystem que se encarga de controlar los inputs.

5. Un GameObject vacío donde se aloja el script de BulletPool

6. Un GameObject vacío donde se aloja el script de MeteorPool

7. El Canvas principal, donde se alojan los siguientes objetos:
    a) El texto de la puntuación
    b) El panel de pausa, con su texto de “Pause” y su botón de “Resume”
    c) El panel de GameOver, con su texto de “You Died!” y su botón de “Restart”

3. Mecánicas:

- Iteración 1:

1. Movimiento: Se ha utilizado un sistema de movimiento en modo tanque, la rotación es lineal y la aceleración funciona con un sistema de fuerzas de empuje, utilizando el método AddForce().

Miguel Ángel
Jiménez San Nicolás
Fundamentos de Videojuegos

2. Mecánica de disparo: Se ha acoplado un generador de Bullets a la nave, donde se disparan balas hacia la dirección que apunta la nave. Para evitar que las balas colisionen con la nave al salir disparadas, se ha añadido una excepción en el collider de la nave para que ignore a los elementos del Layer “Bullet”.

3. Generación de asteroides: Para implementar la generación aleatoria de asteroides se ha acoplado al objeto del espacio un GameObject vacío que contiene el script de generación de asteroides. Estos se generan de manera aleatoria en el rango de la pantalla, con gravedad fija para simular la caída y una altura fija ligeramente superior a la pantalla, para evitar que se vea la generación de dichos asteroides. La tasa de generación de asteroides va incrementando según va pasando el tiempo para que aumente la dificultad.

- Iteración 2(Extras):

1. Fragmentación de asteroides: Con el uso de la herramienta de edición de imágenes GIMP, he recortado el asteroide original para crear dos fragmentos de él. Estos fragmentos salen disparados con dirección de los ángulos de la bisectriz de la trayectoria de la bala. Para lograr esto, cuando una bala colisiona con un asteroide, esta ejecuta un método llamado Dividir(Vector3 bulletDirection), que pasa como argumento su propia dirección. Este método genera dos fragmentos de asteroide en la dirección bulletDirection +45º y -45º en cada fragmento. La fuerza que se aplica es constante con un valor de 5, y se da un efecto rotatorio a los fragmentos con el método AddTorque().

2. Menú de pausa y GameOver: Creando un Canvas y con un script para el menú de pausa he hecho que cada vez que se detecte la tecla “Escape” salga un panel con un botón de Resume(continuar) y un texto de Pause. Adicionalmente, he creado otro panel que aparece cuando la nave colisiona con un asteroide. Este panel tiene fondo rojo y un botón de Restart(reiniciar) y un texto con You Died!. Para evitar el solapamiento de paneles, he añadido una condición al menú de pausa para que solo salga cuando el jugador está vivo, y no cuando esté en el menú de GameOver.

3. Espacio Infinito: Cabe mencionar mi intento de crear un espacio realmente infinito donde la nave se queda quieta en las coordenadas (0,0,0) y es el entorno lo que se movía, pero me ha sido imposible implementarlo ya que había que modificar todo el código y no sabía si iba a quedar realmente bien. Por ello, he implementado que simplemente la nave se teletransporte a sus coordenadas opuestas cada vez que cruce un extremo de la pantalla, evitando la pérdida de la nave por el espacio.

4. Pooling: Para manejar de forma más eficiente la gran cantidad de balas y asteroides que se generan por partida, he aprendido el método de pooling. Al iniciar la partida se crean 30 objetos Bullet y 20 Meteor, esto es gestionado por dos scripts separados llamados BulletPool y MeteorPool. Estos se guardan en un objeto Cola(Queue(FIFO)), y así, cuando una bala desaparece o colisiona con otro objeto, en vez de destruirse para crear otra nueva, que consume muchos recursos, se recicla y vuelve a la pila de balas. Así, los scripts de generación de meteoritos y de

balas solo tienen que hacer un getBullet() o getMeteor() para adquirir un gameObject de la cola. Estos métodos sacan el primer elemento de la cola, lo activan, y lo reintroducen al final de la cola, para después devolverlo con un return. Así los scripts que manejan las colisiones de bullets y meteors reciben el objeto activo, y cuando colisionen en vez de destruirse se desactivan, volviendo a estar disponibles para la cola.

4. Scripts:

He utilizado 8 scripts:

1. Player: Controla el movimiento de la nave, los disparos, y detecta cuando la nave colisiona y muere. Para manejar las interacciones, he utilizado el más reciente InputSystem de Unity, donde primero se declaran y se activan las InputActions para manejarlas durante el script.

2. Bullet: Controla el movimiento y las colisiones de las balas, además de la puntuación y las actualizaciones del texto de puntuación. Intenté implementar un TextMesh pero al final por falta de tiempo utilicé un Text(Legacy).

3. Meteor: Maneja la división de los asteroides cuando colisionan, su fuerza, rotación y dirección (bisectriz de la dirección de la bala), y su correcta desactivación.

4. EnemySpawner: Es el script encargado de generar los meteoritos, con su aleatoriedad y el incremento de dificultad según pasa el tiempo.

5. MeteorPool: Es el script encargado de crear y gestionar el pool de asteroides.

6. BulletPool: Es el encargado de crear y gestionar el pool de las balas.

7. pauseMenu: Es el script que controla el panel de pausa, detectando cuando se pulsa la tecla “Escape” para mostrar el menú, y pausando el juego utilizando Time.timeScale. Tiene en cuenta que la nave debe estar viva para poder activar el menú de pausa, evitando el solapamiento de paneles.

8. GameOverMenu: Es el script que controla el panel de GameOver, que se activa cuando la nave colisiona con un asteroide.